Building Block Recipes

Let's talk about Build Recipes. 🌱 + 💧 + 💩🕳️ + ⏳ => 🌳 Simple equations of resource value flows through a process to form outputs. You've seen them all over the place, in video games, in cookbooks, and when re-reading a Youtube tutorial description for what to pickup from the hardware store. We intuitively understand them, we've grown with them throughout history, but we rarely seem to formalize and universalize them - seemingly preferring highly-complex programming frameworks which only a few tortured souls understand.

But this little pattern is powerful. It might be sufficient to decompose pretty-much every useful process out there into byte-sized, understandable "recipes" with just as high precision as those complex monolith systems. When we use them a little more loosely, with some measure of fudge-factor ("low-fidelity" or "soft" definitions), these napkin-math estimations become capable of describing nearly anything.

I theorize this is one of the most versatile and naturally-understandable ways of describing complex systems, with a strong historic basis, and some very nice decentralized-scaling properties. Build recipes are a lovely protocol for describing complex behavior, and I hope to prove this through a research trial by fire in this Summer of Protocols series, testing their theoretical strength as a model and establishing a meta-protocol to build-up an open public database of build-recipe-based economic pathways.

Secret Protocol Riddle:

🌱🎱🕒

⏰~60s 💻 => 🔑 💻

✍️/🔑: 📢 🔑 ✍️ 🖼️=> ✅

👀/🔑: 📢 🔑 📆0-1 => ✍️ 🖼️ 📆

(Answer below!)

Recipes Are Everywhere Already: The world is rich with them. We can mass-extract and decompose game economies, existing companies, supply chains, Amazon/Alibaba products, Thingiverse models, Wikipedia, old Cooking Recipes, etc etc into tokenized composable recipes. These could then henceforth be analyzed and mapped in relation to all other recipes/products out there, making an economic web of itemized assets and their production processes. This can be as simple as grandma's pie recipe, or as complex as growing food in a post-meteor-impact Earth. With a rich network of data, the pathways fill-in fast!

Fidelity Scaling (Soft => Hard): We need the ability to handle a wide range of problems - some which are defined with very loose definitions (e.g. "Iron Ore + Heat => Iron") and some with much more precision (entire 20-step math-heavy graph of the full iron recipe). By building in the versatility to handle this whole range of fidelity, we're able to use this build recipe format from

start to finish, improving precision/quality as we go. E.g. we might quickly plan macro scale napkin math for a new venture at low fidelity ("Lego" bricks), drill down to medium fidelity for tighter assumptions ("Meccano" pieces - still removing the hard edges of reality, but a very close fit), then we build the final project with precise physics and data, recording exact requirements in the real word. Every representation of this process is useful at different stages, and for different contexts (like clean simple visualization, vs gitty-gritty exact reality)

Handles Black-Box Unknowns: Similarly, when we just don't know how a process works (f***in' magnets!), but we know its inputs and outputs, we can still include it in the model and learn from how it relates to other processes.

Test Framework: We can then improve our knowledge and confidence by defining repeatable trials to test these recipes in the real world (or digital/game world, no reason this same protocol couldn't be used to define game economies too!)

Marketplace for Research: By financially incentivizing this research process to test and verify these production pathways, we can harness the power of the internet marketplace and get the swarm to perpetually improve this model of reality, adding error bounties, rewards, or betting market incentives to the areas most needing accurate precision and mass production capability.

Composable Economy: By building a framework focused on resources and their recipe pathways as the fundamental units - not companies - we can tokenize the resources and recipes themselves, testing and tracking each step directly as they flow through an economy.

Widely Applicable: This creates a self-perpetuating juggernaut of economic and scientific research, improving the available data and performance of all participants in the network. Ideal for game developers, economists, business analysts, and societies forging post-scarcity pathways, even as forces like AI cause us to re-evaluate all our assumptions. With this model, as pathways break and new ones form, we can quickly find new optimal production paths and share them with the network, gaining strength and knowledge from stressors.

Simulation Friendly: Even when we don't have the funds or time to test all these resource flows directly, the nature of this simplified format allows us to easily map unknown values to random variables and run Monte Carlo simulations en-masse to test new models and double-check assumptions. Enough simulations and you can vastly improve confidence before even the first tests in real life.

AI Friendly: Even more promising, we'll be able to plug in the latest developments in AI language models (until we train an economic model directly on this data) to get powerful simulating abilities, as well as rapid generation and extraction of all historical data into this open protocol.

Visualization Friendly: This simple friendly format should quickly birth a wide range of visualization tools: Composable reader UI/UX metaphors like card games, flows, inventories, graphs, and games - all easy to make beautiful/fun when the underlying data is so versatile.

Emoji Friendly: We can easily map resource types to emojis, or better yet - Stable Diffusion (AI) generated icons for minimally-compact representations of resources, providing understanding-at-a-glance. 🍅 🥬 🍄3 => 🥗 These emojis aren't strictly necessary, but they set a good goal for making quick, intuitive representations of recipe equations. No more need for multi-page definitions or brain-melting greek letters. (Besides, AIs will be interpreting most of this to give us plain english explanations on demand)

Protocol Challenges

While this is still an early seed of an idea and liable to shift in definition, its core concept has strong legs and seems to stand up as a model to most challenging conceptual problems, once your brain shakes off old, more-complex patterns of doing things. The test of its merits in the Summer of Protocols will be whether it can hold up under fire from multiple researchers trying to poke holes and apply strange edge-cases. We need this strong theoretical underpinning before attempting any mass-scale protocol development, so I welcome aggressive attempts to break it! Here's some of the weirder seams, where you might smell blood:

Transportation's A Recipe: 🍔 📍_Kitchen => 🍔 📍_Dining_Table_7 (serving burger, low fidelity)
Time's A Recipe: 🌱 💩🕳️ 💧3000L ☀️180 => 🌳 (growing a tree, low fidelity)
Conditional Requirements Are Recipes Too!: ✂ 🐑 => ✂ 🦙 🧵 (shearing sheep requires shears, low fidelity)
Math Ain't *That* Tricky: 🍎 🍌1-3 🍇D6 🍓5σ.2 => 🥗🍌1-1.5 (complex recipe for fruit salad, using ranges (1-3), probability (1D6 dice roll 🎲), and even mean + standard deviation for high-level control of random number generation! This is just a proof of concept that compact visualizations are still doable even with this complex math. But this should be close to all we need for full simulations, at least when assuming normal distributions. Speaking of math…
Matrix Algebra Loves GPUs: Something about just cleverly mapping key/values to hash tables and mass-computing them layer by layer makes a programmer very happy...

First Example Projects

"Specifics First" to hone the model

🏭 **Factory Planning**: Mapping out from low fidelity estimations => high confidence for mass-scale partially-automated factory production (full loop of an economy, including processing and transport). Already have research affiliates creating the economic data in build-recipe formatting
🚑 **Emergency Food Pathways:** Mapping early supply chain pathways for food production (trees => glucose is surprisingly viable even in no-power scenarios). An affiliate project plans a presentation in visual book medium for mass distribution and preservation in worst-case scenarios
🕹️ **3D Game Visualization**: Grid-based Unreal Engine simulator to map recipe resources to 3D models and quickly populate a game board, zoomable down to smaller grids (already mostly built). Potential UE5 plugin!
🌐 **Economic Simulator App**: Run arbitrary rules with goals in mind, compute estimated required/resulting resources over time (also mostly built already!)

Open Ideas for Affiliate Projects

- General Framework for atomizing ALL other protocols using this one (soft <=> hard fidelity support should make this possible, otherwise it's a good signal this model is insufficient)
- Game Designs / Integrations / Mods (we will absolutely be analyzing all the popular build trees from crafting games, and looking at data for new interesting ones)
- Visualization Tools (cards, flows, graphs, grids)
- Stable Diffusion Universal Emoji generator (very doable)
- Unreal Engine Universal 3D Resource Models (AIs will do this by themselves any day now)
- Mass Scraper DB Populator Tools (or just pull from GPT)
- Automated Testing Frameworks (e.g. Robot cooks! coming soon...)
- Crypto-Backed Resource Assets
- Crypto-Backed Production Service (Recipe) Contracts
- Thingiverse Integration (production + assembly)
- Digital Twin Integration (for high fidelity reality mirroring)
- Game-Playing AI (to auto-evaluate our economic models)

Open Research Questions

What *Isn't* A Recipe?: ChatGPT is teaching me Category Theory on this one (using cooking analogies and emojis!) Bad/Good news recipes are basically morphisms, aka functional programming. Excessively generalizable!
How Do We Map Fidelity?: Largely a question of hand-waving, what information matters and what can be practically ignored, and where are the stratums? Fun philosophy topic, hopefully answered by analyzing use cases and existing economic literature/practices.
How Do We Incentivize?: Focusing on narrowing this down to a useful classifier first, but encouraging low-commitment growth really makes it a protocol. Betting markets might satisfy the Bayesians.
Where's Our Complexity Limit?: Can we get by with less than a Turing Complete programming language here? Maybe some things get full contracts while others are simple value flows
How to Deal with Namespace Bloat?: Simple naming of resources/properties goes out the window as you add contexts. Age-old problem. Ranges of fidelities can make it generally true but not precisely true which could be fine, with inference. Hard to have that + classical computing automation though.
What's the Deal With Time?: One of the few less-intuitive seams in this model when mapping to everything. To programmers: this is the difference between functional and imperative programming. Time *could* be seen as a resource, or inherently baked into the process, but when does that fail?

Emergent Commonalities?: Particularly when grouped by low-fidelity, we wonder what unexpected processes have very similar recipes
Language as Recipes?: Sentence: Noun + Noun+adjective ==(Verb + Adverbs)=> Noun+Adjective + Noun... Linguist nerd-snipe here!

Inspirations

- Chemistry/physics equations
- Factorio, Satisfactory, AutomaChef, all the Greats
- Machinations.io => great simulator visualization (though slightly opinionated on time & signaling)
- Scott McCloud's "Understanding Comics" triangle of reality <=> symbolic (recipe) <=> abstraction (fidelity!). True in comic books as in any representational calculus
- BazaarBot: An Open Source Economics Engine
- @Kojak(Jose Antony)'s suggestion of Heines et al "The Tokenization of Everything: Towards a Framework for Understanding the Potentials of Tokenized Assets"

Riddle Answer

🌱🎱🕒: Spring83 (Low Fidelity)
📢: Publisher (Url)
✍: Spring-Signature
🔑: Generated Board Key
🖼️: Board (visual)
📅: DateTime Modified
✍/🔑: PUT/<key>
👀/🔑: GET/<key>

⏰~60s 💻 => 🔑 💻 About 60s to generate a key on a typical computer
✍/🔑: 📢 🔑 ✍ 🖼️ => ✅ To publish (PUT), pass your key and new board to publisher using its signature (✅ designates success)
👀/🔑: 📢 🔑 📅0-1 => ✍ 🖼️ 📅 To view a board (GET), pass your key to publisher to get the board, (optional: only get it if modified since datetime)